2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code) | 979-8-3315-2615-3/25/$31.00 ©2025 IEEE | DOI: 10.1109/LLM4Code66737.2025.00009

CWEVAL: Outcome-driven Evaluation on Functionality and Security of LLM Code Generation

Jinjun Peng, Leyi Cui, Kele Huang, Junfeng Yang, Baishakhi Ray
Department of Computer Science
Columbia University
New York, NY, U.S.A.
{jinjun.peng, angel.c}@columbia.edu, {kele, junfeng, rayb}@cs.columbia.edu

***Abstract*—Large Language Models (LLMs) have significantly aided developers by generating or assisting in code writing, enhancing productivity across various tasks. While identifying incorrect code is often straightforward, detecting vulnerabilities in code is more challenging, especially for developers with limited security knowledge, which poses considerable security risks of using LLM-generated code and underscores the need for robust evaluation benchmarks that assess both functional correctness and security. Current security evaluation benchmarks like CyberSecEval and SecurityEval are hindered by unclear and impractical specifications, failing to assess both functionality and security accurately. To tackle these deficiencies, we introduce CWEVAL, a novel outcome-driven evaluation framework designed to enhance the evaluation of secure code generation by LLMs. This framework not only assesses code functionality but also its security simultaneously with high-quality task specifications and outcome-driven test oracles which provides high accuracy. Coupled with CWEVAL-BENCH, a multilingual, security-critical coding benchmark, CWEVAL provides a rigorous empirical security evaluation on LLM-generated code, overcoming the shortcomings of previous benchmarks. Through our evaluations, CWEVAL reveals a notable portion of functional but insecure code generated by LLMs, and shows a serious inaccuracy of previous evaluations, ultimately contributing significantly to the field of secure code generation. We open-source our artifact at: https://github.com/Co1lin/CWEval.**

***Index Terms*—secure code generation, LLM code generation, benchmark, vulnerability**

I. Introduction

Large Language Models (LLMs) have been extensively used to generate or assist in writing code in recent years [2]–[4], providing developers with substantial productivity gains across a wide range of programming tasks [5]–[9]. Many benchmarks have been developed to track the rapid advancements of LLM in code generation [2], [10], [11]. While most emphasize functional correctness—a key criteria for acceptance by human developers—recent efforts have begun exploring efficiency evaluation, collectively aiming to encompass all critical aspects of benchmarking LLM-generated code [12], [13].

By automating repetitive or complex coding tasks, LLM code generation tools can significantly boost productivity and reduce development time. However, these models also pose potential risks alongside their benefits, particularly when they generate insecure or vulnerable code [14], [15]. Growing evidence underscores the potential security vulnerabilities inherent in LLM-generated code. Prior study [1] demonstrates that LLMs frequently produce insecure code when faced with security-critical scenarios, particularly in situations where even human developers are prone to implementing vulnerable solutions. The challenge is compounded by the fact that while functionally incorrect LLM-generated code can be easily identified and discarded, vulnerabilities embedded within functionally correct code often go unnoticed, especially by non-security experts. This poses significant security risks for systems that integrate such code. To address these concerns, initial efforts have been directed toward *secure code generation*, an approach that enhances existing code generation pipelines to minimize insecure outputs and train models to prioritize secure coding practices, akin to LLM safety alignment in natural language tasks.

Building on these concerns, several benchmarks and tools have been developed to evaluate the security of LLM-generated code. While they have been utilized in prior empirical studies and security training efforts, they face three key limitations that undermine their ability to provide accurate and consistent security assessments of LLM code generation:

1) **Poorly defined specifications:** Popular benchmarks for evaluating functional correctness typically provide well-structured instructions to guide LLMs in generating accurate code. For instance, each example in the HumanEval benchmark includes a function signature that defines inputs and outputs, a natural language docstring describing the expected behavior, and example test cases to eliminate ambiguities. In contrast, existing benchmarks for secure code generation often lack such clear and detailed specifications, making it challenging for LLMs to produce secure and functional code. For example, CyberSecEval [16], [17] uses vulnerable code automatically mined from open-source repositories, and offers limited guidance by providing only a fixed number of preceding lines as context or a natural language summary of the vulnerable function. The summary, being generated by an LLM, is often vague and lacks accuracy guarantees. While SecurityEval [1] offers relatively complete contexts by designing self-contained scenarios, its descriptions are usually limited to one-sentence high-level comments, making it difficult even for humans to discern the precise functionality required. These poorly

979-8-3315-2615-3/25/$31.00 ©2025 IEEE
DOI 10.1109/LLM4Code66737.2025.00009

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on December 16,2025 at 21:12:32 UTC from IEEE Xplore. Restrictions apply.

Fig. 1: (1) The CodeQL rule checking "incomplete URL substring sanitization (CWE-020)" looks for certain insecure sanitization methods including `in`, `startswith` and `endswith`. This rule is used in SecurityEval [1] for the coding task shown in Fig. 2 (at the upper-right corner). (2) Two vulnerable implementations are successfully caught by this rule. (3) However, with only slight differences, two other insecure code are not reported and considered as safe ones in previous evaluations (false negatives). (4) Plus, a secure implementation can also be wrongly flagged as vulnerable (false positives).

defined specifications can lead to an underestimation of security risks—in extreme cases, an LLM might produce a no-op solution to avoid vulnerabilities, which ensures security but does not follow the user intention, which is entirely impractical for real-world applications.

2) **Infeasibility of rigorous functionality evaluation:** Due to vague specifications, complex setup requirements for external dependencies (e.g., CyberSecEval samples), and the absence of comprehensive test cases, current benchmarks fail to assess the functionality of LLM-generated code in security-critical contexts. As a workaround, previous research in secure code generation has employed separate benchmarks to evaluate functionality and security. For example, studies have used conventional functionality benchmarks to measure the functional capability of models fine-tuned with security objectives [18], [19]. However, tasks designed to evaluate functionality often emphasize algorithmic solutions and do not typically address security-critical operations such as file handling, process creation, or sensitive data processing. This discrepancy creates a gap, enabling models adept at algorithmic coding but lacking in generating secure and functional code to score well on both benchmarks, which obscures a true evaluation of the *alignment tax* [20] (as supported by evaluation results in Section V-C3).

3) **Instability of security evaluation:** All existing benchmarks rely on static analysis tools to identify vulnerabilities in LLM-generated code. While static analysis has the advantage of being automatic and scalable, it cannot provide stable and accurate feedback on security. For instance, only less than a third (562/1916) of the vulnerable samples included in CyberSecEval can be reproduced, i.e. still being flagged as vulnerable by its associated static analysis tool, as the tool struggles to operate on the provided incomplete code snippets with syntax errors and missing dependencies. For SecurityEval, despite the design of self-contained scenarios and the usage of CodeQL, an industry-leading static analysis tool by GitHub, its evaluation still suffers from both false negatives and false positives due to the inability of static analysis to flexibly model different semantic-equivalent implementations, as shown in Fig. 1.

Our proposal. Driven by the insights highlighted earlier, we introduce CWEVAL, an evaluation framework designed to overcome the current shortcomings in assessing secure code generation. CWEVAL leverages human-verified, high-quality, security-critical coding tasks that come with comprehensive specifications, test oracles for both functionality and security, and reference implementations in both insecure and secure forms, enabling a thorough assessment of LLMs' security

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on December 16,2025 at 21:12:32 UTC from IEEE Xplore. Restrictions apply.

capabilities in code generation.

Furthermore, we have developed CWEVAL-BENCH, a multilingual security-critical coding benchmark based on CWEVAL, to empirically investigate the security attributes of code generated by leading LLMs. CWEVAL offers several advantages over previous evaluation methods, outlined as follows:

- **Full reproducibility:** We design self-contained coding scenarios, each manually verified by expert programmers. For each coding task, we provide a sample secure solution and at least one vulnerable counterpart. The vulnerable example can establish the significance of security for a task, verify the existence of a vulnerability, and demonstrate that this vulnerability can be mitigated without affecting the overall functionality.
- **Clear specification:** For each coding task, we provide detailed specifications that match the high standards of popular functionality benchmarks. These include a natural language description of the required functionality, a function signature detailing the exact inputs, outputs, and data types, and example test cases for further clarification. This comprehensive approach facilitates LLMs' understanding of our expectations.
- **Simultaneous functionality and security evaluation:** We create two types of test oracles to simultaneously assess the functionality and security of code generated by LLMs. An optimal LLM should generate responses that successfully pass all tests in both categories for the same task, demonstrating not only the ability to handle security-critical tasks but also to do so with proper security awareness and implementation.
- **High accuracy and flexibility:** We are the first to use outcome-driven test oracles to simultaneously evaluate the functionality and security of code generated by LLMs. This approach monitors the dynamic properties of LLM-generated code, allowing it to adaptively and reliably manage the inherent diversity of code implementations. It offers more accurate evaluations of the security of LLM-generated code compared to traditional static analysis.

Contributions of our work include:

- **Dimension:** We propose CWEVAL, the first evaluation method to our knowledge that simultaneously evaluates both functionality and security of LLM-generated code on the same problem set, offering a rigorous testbed for future efforts on secure code generation.
- **Technique:** We design coding tasks, test oracles and reference solutions of both types for cross-checking, ensuring full reproducibility and validity. We implement various test oracles to capture dynamic properties of LLM-generated code to accurately assess both their functionality and security.
- **Benchmark:** We open-source the complete benchmark suite CWEVAL-BENCH (https://github.com/Co1lin/CWEval), which consists of the whole evaluation pipeline and 119 high-quality security-critical coding tasks covering 31 CWEs across 5 popular programming languages. CWEVAL-BENCH is designed to be easily expandable through continuous development.
- **Study:** With CWEVAL-BENCH, we comprehensively evaluate four popular LLM families and show empirical results regarding the security risks of LLM code generation and inaccuracy of previous evaluations.

II. Related Work

Current benchmarks designed for evaluating code LLMs primarily focus on functional correctness and general-purpose code generation, with limited emphasis on assessing the security or detecting vulnerabilities in the generated code. For example, HumanEval [10] is a widely used dataset for assessing the functional correctness of LLM generated code [4], [10], without any consideration on the security aspects. While other datasets and benchmarks [2], [13], [21], [22] extend to various aspects of evaluating LLM code generation, none of them specifically address the security evaluation.

Limit current works are focused on evaluating the security of code generated by LLMs. SecurityEval [1] introduces a dataset that evaluates code generation security, manually covering vulnerabilities across 40 CWE-related categories. However, its poor task specifications, lack of functionality test cases and the reliance on static analysis often lead to unreliable results. Similarly, the CyberSecEval dataset, part of the PurpleLlama benchmarks, also evaluates secure code generation [16], [17]. However, its non-self-contained and noisy data also suffer from the same issues.

Other existing efforts toward secure code generation include SVEN [18] and SafeCoder [23], which aim to improve secure code generation by fine-tuning LLMs with specially designed security-related learning objectives. However, these efforts mainly rely on SecurityEval and CyberSecEval with minor changes. The unresolved limitations in quality, reproducibility and accuracy in existing benchmarks underscore the need for more comprehensive and reliable benchmarks for evaluating secure code generation.

III. Our Method: CWEVAL

Problem formulation. As shown in Fig. 2, there are three key elements at the heart of CWEVAL, the coding task specification $\mathcal{P}$, the code implementation $\mathcal{I}$ to fulfill the task, and the test oracles $\mathcal{T}$ to verify the functionality and security of the implementation producing evaluation results *pass* or *fail*. Specifically, we denote functional correctness as f^+ (correct) and f^- (incorrect), and security property as s^+ (secure) and s^- (insecure), respectively. We label test oracles corresponding to a specification $\mathcal{P}$ for functionality evaluation as $\mathcal{T}_{\mathrm{f}}^{\mathcal{P}}$ and the ones for security testing as $\mathcal{T}_{\mathrm{s}}^{\mathcal{P}}$. Then, we can denote a functionally correct and secure implementation to a specification $\mathcal{P}$ as $\mathcal{I}_{\mathrm{f}^+,\mathrm{s}^+}^{\mathcal{P}}$, where $\mathcal{T}_{\mathrm{f}}^{\mathcal{P}}(\mathcal{I}_{\mathrm{f}^+,\mathrm{s}^+}^{\mathcal{P}}) \equiv \textit{pass}$ and $\mathcal{T}_{\mathrm{s}}^{\mathcal{P}}(\mathcal{I}_{\mathrm{f}^+,\mathrm{s}^+}^{\mathcal{P}}) \equiv \textit{pass}$; similarly, we also have $\mathcal{I}_{\mathrm{f}^+,\mathrm{s}^-}^{\mathcal{P}}$, $\mathcal{I}_{\mathrm{f}^-,\mathrm{s}^+}^{\mathcal{P}}$ and $\mathcal{I}_{\mathrm{f}^-,\mathrm{s}^-}^{\mathcal{P}}$. The goal of CWEVAL is to rigorously assess both the functionality and security of an implementation $\mathcal{I}^{\mathcal{P}}$ produced by an LLM for a given security-critical coding task $\mathcal{P}$. Our evaluation demonstrates the LLM's ability to

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on December 16,2025 at 21:12:32 UTC from IEEE Xplore. Restrictions apply.

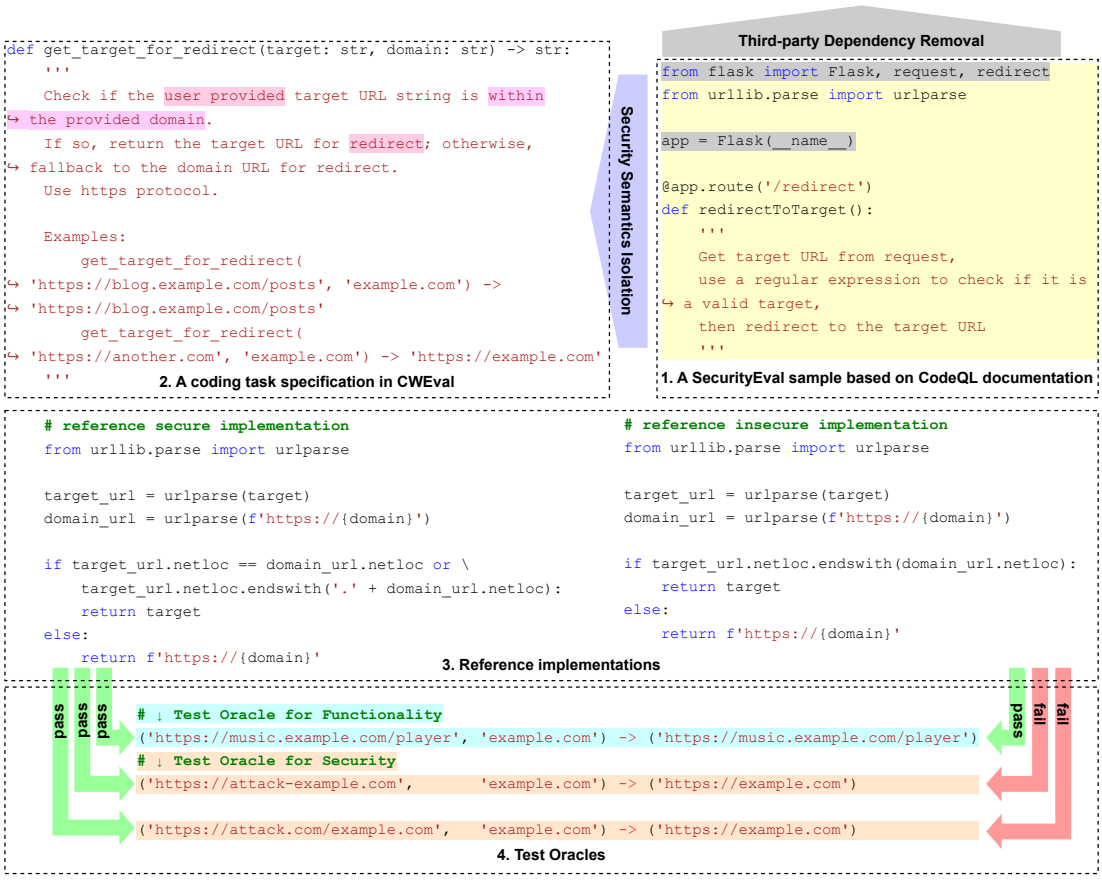

Fig. 2: Inspired by documentations about CWEs, CWEVAL consists of coding tasks with three components: specifications (2), reference implementations (3) and test oracles (4). Compared to previous benchmarks like SecurityEval [1], our coding task designs are isolated from third-party dependencies as much as possible. Our specifications are more definite and ensure the existence of security-related semantics (a user provided URL will be used to do redirecting, so proper sanitation is needed here). Test oracles for functionality and security evaluations are included. For each task, the secure reference implementation can pass all test oracles, while the functional but insecure reference implementation can only pass functionality test oracles and fail on security test oracles.

securely execute security-critical tasks for LLM developers, and highlights the security risks associated with accepting functionally correct but potentially vulnerable code from the perspective of programmers. Below we detail the design of each element to show how we achieve these objectives.

A. Coding Task Specifications

When designing coding tasks and their specifications, we impose the following three requirements.

- **Security-semantics existence:** Most coding tasks in functionality evaluation benchmarks are irrelevant to any security-critical operations, which makes it almost impossible to induce vulnerable code. To evaluate how well LLMs can securely fulfill tasks having potential vulnerability risks, we require that some security-related semantics should exist in the specification, by either the implicit nature of the code behaviors (e.g. asking for file operations) or the explicit definition in the natural language (e.g. a variable is user-provided). Otherwise, there is no distinction between secure and insecure. Though previous designs consider the same factor, they sometimes failed to make this existence clear (e.g. vague about whether a value comes from an user-input).
- **No security-awareness leakage:** We intentionally avoid leaking any security-awareness to LLMs in the specification, so as to simulate the most common but risky practical scenario where the user of the LLMs has little knowledge or carefulness on vulnerability issues. We avoid explicit hint or instructions related to security in both code and natural languages. For example, the keywords "safe" or "unsafe" in variable names or directly instructing the LLMs to perform a task safely, which are sometimes found in previous security evaluation benchmarks, are totally avoided.
- **Expectation unambiguity:** As a benchmark aiming at the security evaluation of LLM-generated code, our focus is not to assess the LLMs' functional capacity to handle complex tasks. Instead, the functionality test oracles

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on December 16,2025 at 21:12:32 UTC from IEEE Xplore. Restrictions apply.

are designed to gauge the potential *alignment tax*—the decrease in functional performance and utility of LLMs as security measures are intensified, akin to safety alignment in natural language processing tasks. Consequently, it's crucial that our functionality specifications are clear and straightforward, enabling LLMs, prior to any security-specific training, to pass the functionality tests with high likelihood. If this clarity is not achieved, we risk conflating the LLMs' failure to understand and complete the task with their potential refusal to execute tasks due to security alignment, which would obstruct our ability to identify the effects of the latter.

B. Test Oracles

For functionality evaluation, we adhere to the established practice in existing benchmarks by specifying the expected values for input and output pairs. For security evaluation, while we still verify the output or return values, we also assess additional properties such as the time cost (detecting DoS vulnerability), the memory access validity (detecting various memory-related vulnerabilities in low-level languages like C), and the side-effect or integrity of data (detecting SQL injection vulnerability). By expanding the feature set of program runtime behavior to capture, our test oracles evaluate an implementation with more dimensions, enhancing the measurement of program security.

The key difference between our approach and previous security benchmarks is that we are *outcome*-driven and do not rely on any static analysis tools. The hardcoded rules for static analysis struggle to model the behaviors of diverse low-level implementations from a higher semantic level, leading to both false positives and false negatives (as shown in Fig. 1). Instead, our methodology employs dynamic analysis, focusing on defining and observing the secure and insecure *outcomes* of code execution rather than modeling how the code is written in specific patterns. This approach provides greater stability and robustness. Besides, the independence from specific tool makes our test oracles language-agnostic, enabling easier multilingual support (in Section IV).

C. Reference Implementations

High-quality specifications and test oracles, developed in accordance with our established guidelines, already enable a robust evaluation of secure code generation. However, to maintain the integrity of our evaluation, we also develop reference implementations, denoted as $(\mathcal{I}_{\text{ref}})^{\mathcal{P}}_{\text{f}^+,\text{s}^-}$ and $(\mathcal{I}_{\text{ref}})^{\mathcal{P}}_{\text{f}^+,\text{s}^+}$. The former, $(\mathcal{I}_{\text{ref}})^{\mathcal{P}}_{\text{f}^+,\text{s}^-}$, passes all functionality tests but fails on at least one of the security tests, proving the existence and reproducibility of potential security issues in a functional implementation for $\mathcal{P}$ that is very likely to be accepted by programmers. The latter, $(\mathcal{I}_{\text{ref}})^{\mathcal{P}}_{\text{f}^+,\text{s}^+}$, passes both all functionality tests and security tests, showing that there is an implementation which meets the desired functionality while also being free from vulnerabilities. Apart from enabling the cross-check with coding task specifications and test oracles, reference implementations also lay a foundation for future extensions, such as the differential testing and test oracle augmentation in EvalPlus [11].

IV. Our Benchmark: CWEval-bench

To realize the CWEval framework, we build CWEval-bench, a high-quality benchmark suite for evaluating secure code generation. Our dataset creation process follows the steps below:

1) **Coding tasks design:** Like previous security evaluation benchmarks, we utilize CWE-related documentations by leading organizations [24], [25], to guide the design of our coding tasks. Each task is crafted to be self-contained, ensuring contextual completeness and facilitating straightforward evaluation. A notable difference of CWEval-bench from earlier approaches is our emphasis on *security semantics isolation*, which aims to minimize or eliminate dependencies on third-party libraries, as the example shown in Fig. 2. This isolation maintains the essential security-critical semantics of the tasks while making them independent of specific external libraries. This not only enables us to assess LLMs' understanding of fundamental security principles but also simplifies the continued extension for multilingual support.
2) **Specifications writing:** For each designed coding task, we write specifications meeting all three requirements in Section III-A, in the form of function signature, natural language docstring, and optional example input and output pairs for further disambiguation. We ensure that each coding task inherently includes security-critical programming behaviors, or we explicitly define the semantics to distinguish between vulnerable and secure implementations, such as the example in Fig. 2. We avoid any explicit instruction for LLM on noticing the requirement on security to prevent security-awareness leakage. In addition, we test our specifications with several common LLMs to see if they can be easily understood, and perform iterative refinement if needed.
3) **Test oracles and reference solutions development:** We follow the same way as HumanEval [10] to write test oracles for functionality evaluation. For security evaluation, we do not always only use output/return values as the oracle. Thus, we setup corresponding tools or testing logics and specify expected secure and insecure outcomes. For instance, for time cost measurement detecting DoS vulnerabilities, we set timeout for running a given implementation as the oracle to see if the implementation can exit gracefully within the time limit. As another example, for out-of-bounds access, one of the C-specific vulnerability, we compile the implementation with address sanitizer and see if it reports any such error during execution as the testing oracle. For reference solutions, we study the principle of the vulnerabilities and then implement a functional and secure version and at least one functional and insecure version while

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on December 16,2025 at 21:12:32 UTC from IEEE Xplore. Restrictions apply.

making sure they are compatible with the test oracles as specified in Section III-C.

4) **Multilingual evolution:** Since the design of our tasks and specifications are isolated from specific language features and third-party libraries as much as possible, we can easily translate a task along with its specifications, test oracles and reference solutions implemented in one language to another, to evaluate LLMs' security capability more comprehensively. We first use LLMs to do automatic translation, and then manually go over each with necessary refinement to make sure their validity. Tasks remain valid in all supported languages form a core testing set, and other tasks serve as language-specific ones to further cover language-aware vulnerabilities.

Up to the time of writing this paper, CWEVAL-BENCH consists of 119 high-quality security-critical coding tasks along with their specifications, test oracles and reference implementations. It covers 31 CWE types, spans 5 popular programming languages and includes 11 C-specific tasks of vulnerabilities related to memory, serving as an out-of-the-box and stable benchmark for secure code generation. While the total task count is currently limited due to the limited human resources we have, CWEVAL-BENCH can be easily augmented by adding more CWE cases covered in security advisories or evolving to more programming languages, in manual or potential automatic ways.

V. EVALUATION

Using CWEVAL-BENCH, we perform a thorough benchmarking for several popular LLMs, specifically aiming to answer the following research questions:

RQ.1. How do popular LLMs perform on CWEVAL-BENCH? Particularly, how large is the gap between functional correctness and security?

RQ.2. Can larger models achieve better performance on CWEVAL-BENCH, showing higher capability of writing secure *and* functional code?

RQ.3. Can prompting with security instruction and existing fine-tuning techniques help LLMs generate more secure code? How does it affect models' functionality performance?

A. Metrics

We evaluate the following two metrics to benchmark LLM secure code generation, which are adaptations of the widely used pass@k metric in functionality evaluation [10].

- func@k follows the same definition as pass@k, indicating how likely any implementation out of k LLM-generated implementations is functionally correct, i.e. passing all functionality test oracles.
- func-sec@k evaluates both functionality and security, indicating how likely any implementation out of k LLM-generated implementations is functionally correct *and* secure, i.e. passing both functionality and security test oracles.

All the two metrics above are calculated in the same way as pass@k, i.e. by the unbiased estimator. For example, func-sec@$k = \mathbb{E}_{\text{Problems}}[1-\frac{\binom{n-c}{k}}{\binom{n}{k}}]$, where n is the total number of sampled implementations, $k \leq n$, c is the number of implementations that are both functional and safe.

B. Setup

Model selection. We mainly study four popular LLMs, comprising three commercial models and one open-source model, including GPT-4o mini (`gpt-4o-mini-2024-07-18`), Claude 3.5 Haiku (`claude-3-5-haiku-20241022`), Gemini 1.5 Flash (`gemini-1.5-flash-002`) and Llama 3.1 70B Instruct. For RQ.2, we also study their variants of different sizes, including GPT-4o (`gpt-4o-2024-08-06`), Claude 3.5 Sonnet (`claude-3-5-sonnet-20241022`), Gemini 1.5 Pro (`gemini-1.5-pro-002`), Llama 3.1 8B Instruct, and Llama 3.1 405B Instruct.

Experimental settings. Similar to typical settings for functional evaluation in prior works [10], [11], for each model in RQ.1, we perform: (1) random sampling to generate $n = 100$ program samples for each of the four temperature settings (0.2, 0.4, 0.6, 0.8), with showing the best-performing $\cdot@k$ for $k = 1, 10, 50$; and (2) greedy-search decoding, with showing the pass rate of the only deterministic sample as $\cdot@k^*$. Due to limited budget, for RQ.2 and RQ.3 we only evaluate random sampling ($n = 100$) with temperature 0.8.

C. Results

1) RQ.1. Performance of Leading LLMs on CWEVAL-BENCH*:* Fig. 3 shows the evaluation results of five LLMs. We observe that for all LLMs, there is a significant performance gap between only functionality pass rate and pass rate requiring both functionality and security. From func@10 to func-sec@10, the performance drops around 30% across all models, with the maximum 35.79% observed on Gemini 1.5 Flash. This shows that in security-critical coding scenarios, LLMs often generate functional but insecure code with vulnerability issues, which is very likely to be ignored by developers and introduce serious potential risk. Additionally, models that exhibit higher func@k tend to also achieve better scores on func-sec@k, which is logical since the latter requires functional correctness as a prerequisite. However, there could also be counter-examples: Llama 3.1 70B Instruct performs better than Claude 3.5 Haiku in terms of func@10, but the latter achieves higher score on func-sec@10.

2) RQ.2. Performance of larger LLMs on CWEVAL-BENCH*:* Table I shows the comparison between the performance of larger version LLMs and their smaller versions within the same model family. It shows that larger models almost always achieves higher func-sec@k. Additionally, for the GPT-4o family and Gemini 1.5 family, while the differences on func@k are small and even the smaller models perform slightly better, the differences on func-sec@k are larger, which reveals potential but critical neglected differences between larger models and their smaller alternatives in the aspect of security awareness and capability.

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on December 16,2025 at 21:12:32 UTC from IEEE Xplore. Restrictions apply.

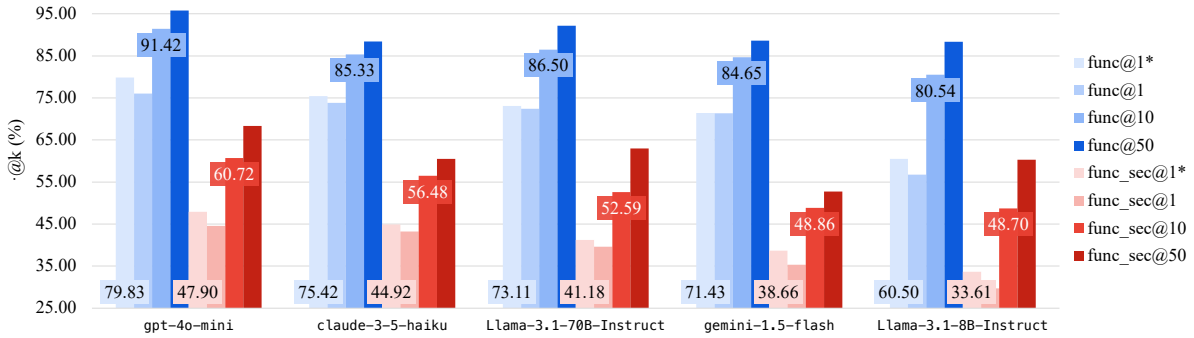

Fig. 3: Evaluating LLMs on CWEVAL-BENCH. Best performing results among all temperature settings are presented. Results of greedy decoding are labeled as func@1* and func-sec@1*. Results of func@10 and func-sec@10 are labeled at upper positions. Results of func@1* and func-sec@1* are labeled at the bottom.

TABLE I: Comparison between larger LLMs with smaller ones of the same model family. Results of larger LLMs are filled with the blue background. We do random sampling with $n = 100$ and temperature 0.8.

	func@1	func@10	func@50	func-sec@1	func-sec@10	func-sec@50
GPT-4o	**80.81**	90.71	93.45	**50.21**	**65.33**	**70.7**
GPT-4o mini	75.43	**91.42**	**95.76**	44.54	60.28	67.68
Gemini 1.5 Pro	68.99	83.25	87.6	**38.09**	**53.08**	**59.42**
Gemini 1.5 Flash	**70.71**	**84.41**	**88.32**	35.31	47.81	51.77
Claude 3.5 Sonnet	**78.16**	**91.36**	**94.73**	**46.69**	**59.2**	**63.6**
Claude 3.5 Haiku	73.81	85.33	87.9	43.2	56.48	60.5
Llama 3.1 405B Instruct	69.25	**88.93**	**93.45**	36.18	**53.7**	**64.39**
Llama 3.1 70B Instruct	**71.84**	85.21	92.11	**39.58**	52.07	61.78
Llama 3.1 8B Instruct	52.62	80.54	88.34	26.53	48.7	60.31

TABLE II: Exploring to improve the performance of LLMs on CWEVAL-BENCH with security instruction prompting (w/ security instr.) or SafeCoder fine-tuning [23]. We do random sampling with $n = 100$ and temperature 0.8.

	func@1	func@10	func@50	func-sec@1	func-sec@10	func-sec@50
GPT-4o mini	**75.43**	**91.42**	95.76	44.54	60.28	67.68
GPT-4o mini w/ security instr.	75.23	90.78	**96.30**	**48.21**	**64.12**	**72.24**
Claude 3.5 Haiku	**73.81**	**85.33**	87.90	43.20	56.48	60.50
Claude 3.5 Haiku w/ security instr.	71.64	84.31	**88.28**	**47.10**	**66.28**	**72.56**
Gemini 1.5 Flash	**70.71**	**84.41**	**88.32**	35.31	47.81	51.77
Gemini 1.5 Flash w/ security instr.	68.24	81.32	86.55	**36.15**	**48.97**	**54.61**
Llama 3.1 70B Instruct	**71.84**	**85.21**	92.11	**39.58**	52.07	61.78
Llama 3.1 70B Instruct w/ security instr.	65.79	83.95	**92.39**	36.92	**53.47**	**65.85**
CodeLlama-7b-hf	**30.77**	**68.82**	**81.72**	**13.76**	**40.74**	**58.41**
CodeLlama-7b-hf w/ SafeCoder [23]	15.35	33.75	41.80	7.37	18.26	24.24

3) RQ.3. Exploring to Improve the Performance on CWEVAL-BENCH*:* Table II shows how the performance of LLMs on CWEVAL-BENCH changes with security instruction prompting and security-focus supervised fine-tuning. For prompting, here we only try the simplest way, by adding the instruction "Your code should be secure and should NOT contain any vulnerability" into the prompt. We do not add more informative instructions or CWE-specific instructions, with the purpose of evaluating the existing internal security capability of LLMs with the simplest reminder. Results show that it leads to improvements on func-sec@k for almost all LLMs, and only possible slight decrease on func@k. Improvements for GPT-4o mini and Claude 3.5 Haiku are more significant, up to 9.8% on func-sec@10 for Claude 3.5 Haiku.

For security-focus supervised fine-tuning, we evaluate CodeLlama-7b-hf and its fine-tuned version by SafeCoder [23] (with their released fine-tuned checkpoint). In SafeCoder's evaluation, that is evaluating models' functionality with typical functionality benchmarks and evaluating security with previous security benchmarks separately, the fine-tuned CodeLlama-7b-hf generates much more secure code without loss on functionality. On CWEVAL-BENCH, however, the SafeCoder version shows a significant functionality degradation. The SafeCoder version also generates much

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on December 16,2025 at 21:12:32 UTC from IEEE Xplore. Restrictions apply.

less both functional and secure code than the original base model. This is possibly due to our assumption and also the motivation to propose CWEVAL—the model may learn to avoid generating any security-sensitive code to be securer (though keeps its functionality on other tasks). In practical, it hinders its helpfulness on coding such operations and blocks its acceptance by developers, but separate benchmarks for functionality and security evaluation can fail to capture this issue (a kind of *alignment tax*). Instead, CWEVAL-BENCH evaluates both properties simultaneously, forcing the model to be both functional and secure in security-related coding to achieve a high func-sec@k, which distinguishes learning secure coding practice from learning to avoid security-related coding at all, and thus offers a more comprehensive and rigorous evaluation for secure code generation.

VI. Conclusion and Future Work

This paper presents a comprehensive approach to evaluate both the security and functionality of LLM code generation through the design of the CWEVAL framework and the development of CWEVAL-BENCH. CWEVAL and CWEVAL-BENCH offer a novel simultaneous evaluation approach on high-quality security-critical coding tasks, enabling a more accurate and rigorous assessment of LLM-generated code, addressing limitations observed in current evaluation methodologies. Through empirical evaluation, we reveal the significant gap between writing functional code and writing functional and secure code with several leading LLMs. In addition, we identify a previously ignored but severe pitfall of existing evaluations that use separate tasks for evaluating functionality and security. Potential future work includes automating the process of benchmark creation and expansion to enhance the scalability and efficiency of CWEVAL.

References

[1] M. L. Siddiq and J. C. Santos, "Securityeval dataset: mining vulnerability examples to evaluate machine learning-based code generation techniques," in *Proceedings of the 1st International Workshop on Mining Software Repositories Applications for Privacy and Security*, 2022, pp. 29–33.

[2] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le *et al.*, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[3] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[4] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "Codegen: An open large language model for code with multi-turn program synthesis," *arXiv preprint arXiv:2203.13474*, 2022.

[5] Z. Sun, Q. Zhu, Y. Xiong, Y. Sun, L. Mou, and L. Zhang, "Treegen: A tree-based transformer architecture for code generation," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 05, 2020, pp. 8984–8991.

[6] A. Svyatkovskiy, S. Lee, A. Hadjitofi, M. Riechert, J. V. Franco, and M. Allamanis, "Fast and memory-efficient neural code completion," in *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. IEEE, 2021, pp. 329–340.

[7] S. Kim, J. Zhao, Y. Tian, and S. Chandra, "Code prediction by feeding trees to transformers," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 150–162.

[8] Y. Gao and C. Lyu, "M2ts: Multi-scale multi-modal approach based on transformer for source code summarization," in *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension*, 2022, pp. 24–35.

[9] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 1482–1494.

[10] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[11] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[12] A. Shypula, A. Madaan, Y. Zeng, U. Alon, J. Gardner, M. Hashemi, G. Neubig, P. Ranganathan, O. Bastani, and A. Yazdanbakhsh, "Learning performance-improving code edits," *arXiv preprint arXiv:2302.07867*, 2023.

[13] J. Liu, S. Xie, J. Wang, Y. Wei, Y. Ding, and L. Zhang, "Evaluating language models for efficient code generation," *arXiv preprint arXiv:2408.06450*, 2024.

[14] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of github copilot's code contributions," in *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022, pp. 754–768.

[15] M. L. Siddiq, S. H. Majumder, M. R. Mim, S. Jajodia, and J. C. Santos, "An empirical study of code smells in transformer-based code generation techniques," in *2022 IEEE 22nd International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 2022, pp. 71–82.

[16] M. Bhatt, S. Chennabasappa, C. Nikolaidis, S. Wan, I. Evtimov, D. Gabi, D. Song, F. Ahmad, C. Aschermann, L. Fontana *et al.*, "Purple llama cyberseceval: A secure coding benchmark for language models," *arXiv preprint arXiv:2312.04724*, 2023.

[17] M. Bhatt, S. Chennabasappa, Y. Li, C. Nikolaidis, D. Song, S. Wan, F. Ahmad, C. Aschermann, Y. Chen, D. Kapil *et al.*, "Cyberseceval 2: A wide-ranging cybersecurity evaluation suite for large language models," *arXiv preprint arXiv:2404.13161*, 2024.

[18] J. He and M. Vechev, "Large language models for code: Security hardening and adversarial testing," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023, pp. 1865–1879.

[19] J. He, M. Vero, G. Krasnopolska, and M. Vechev, "Instruction tuning for secure code generation," *arXiv preprint arXiv:2402.09497*, 2024.

[20] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, "Training language models to follow instructions with human feedback," *Advances in neural information processing systems*, vol. 35, pp. 27 730–27 744, 2022.

[21] D. Hendrycks, S. Basart, S. Kadavath, M. Mazeika, A. Arora, E. Guo, C. Burns, S. Puranik, H. He, D. Song *et al.*, "Measuring coding challenge competence with apps," *arXiv preprint arXiv:2105.09938*, 2021.

[22] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, "Livecodebench: Holistic and contamination free evaluation of large language models for code," *arXiv preprint arXiv:2403.07974*, 2024.

[23] H. Su, J. Niu, X. Liu, and M. Atiquzzaman, "Safecoder: A machine-learning-based encoding system to embed safety identification information into qr codes," *Journal of Network and Computer Applications*, vol. 227, p. 103874, 2024.

[24] "Cwe - about cwe," https://cwe.mitre.org/about/index.html, (Accessed on 11/18/2024).

[25] "Codeql documentation," https://codeql.github.com/docs/, (Accessed on 11/18/2024).

Authorized licensed use limited to: Atlantic Technological University (ATU). Downloaded on December 16,2025 at 21:12:32 UTC from IEEE Xplore. Restrictions apply.